Society of Petroleum Engineers

SPE-229198-MS

When to Retrain the Models of AI-Based Raster Digitization Workflow?

R. Srivastava and O. A. Gune, SLB; P. R. Mangsuli, Independent Consultant

Abstract

Rasters logs are powerful source of information for exploration and production of oil and gas. Traditionally these raster logs were printed on the paper, later scanned, and stored on computers. Although present on a computer, these scanned raster logs cannot be treated as a digital data as they cannot be directly consumed or processed. Therefore, a large volume of legacy raster data is still not truly digital. Recently, Artificial Intelligence (AI) based raster digitization tools have been developed to convert the scanned raster logs into true digital format such as LAS or CSV. Typically, the raster digitization system includes various machine learning (ML) / deep learning (DL) models to process and digitize various parts of raster images. For example, raster segmentation model is used to segment out log headers, log track, and depth track area of the raster logs. A separate DL model is used to process each of these three areas. For example, log header processing model extracts the log metadata, depth track processing model extracts the depth information, and curve segmentation model extracts the log curve from the log track. Finally, outputs of all these DL models are consolidated to provide raster data in digital format such as LAS or CSV file. Such AI-based raster digitization system looks promising when tested on the data which is similar in distribution to the data on which AI models are trained. However, the performance of such system drops when test data is different in distribution than the train data. It is well known that the performance of DL model on the test data can be improved by retraining or fine-tuning DL model on (part of) the test data. While retraining helps in improving the model performance, retraining of AI-based raster digitization system is a non-trivial task. This is because typical raster digitization system is complex as it involves multiple ML/DL models in cascade or in parallel. If one wants to retrain the raster digitization system, it is not clear which model(s) of the raster digitization system to retrain. Moreover, if deployed in production, users of such AI-based raster digitization system are agnostic to internal mechanisms of DL models. Hence, there is a necessity of an automated workflow which can assess the performance of individual DL models of raster digitization system on the batch data and recommends the retraining of one or more DL models. The work presented in this paper addresses the above question of when to retrain model(s) of raster digitization system. This work provides a novel, fully automated, and robust inference time mechanism to identify the ML/DL models which require retraining. The proposed novel approach achieves an outstanding performance of 66%, 76.1%, and 81.2% (raster segmentation retraining decision classifier), 82.3%, 91.7%, and 86.9% (log header segmentation retraining decision classifier), and 75.6%, 75.1%, and 77% (curve segmentation retraining

decision classifier) in accurately detecting the scenarios which require retraining for raster segmentation, log header segmentation, and curve segmentation models, respectively, of raster digitization system.

Introduction

The oil and gas rasters are a vital source of information for subsurface exploration and production. Traditionally, these rasters were present in the form of paper. Later, these raster logs were scanned and stored as scanned files (PDFs or images) on digital computers. Figure 1 shows an example of a raster log. A raster log contains three important regions: the log track area, the log header area, and the depth track area. The log track area contains the grid and the curves corresponding to the log properties (such as Gamma Ray, resistivity) which are recorded. The depth track area contains information about the depths along which the property is recorded. Finally, the log header area contains metadata such as the property name, scale, unit, and line style used to plot the log curve.

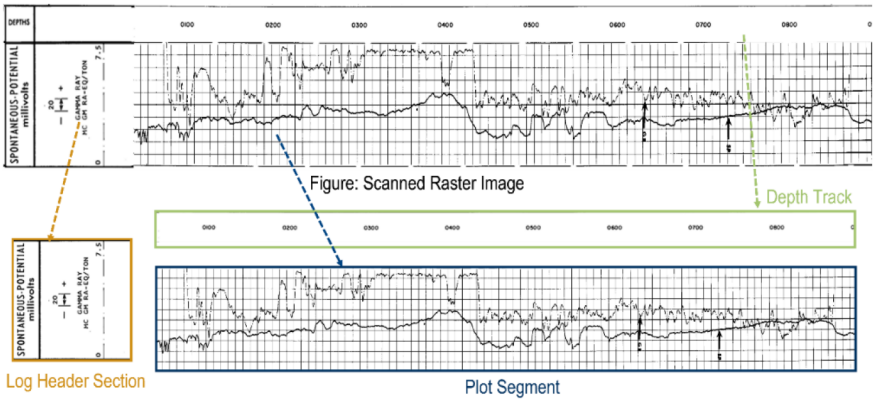

Figure 1—An example illustrating important regions in a raster: 1) Plot segment/ log track, 2) Log header region, 3) Depth track region. Best viewed in colors.

The scanned raster files, as shown in Figure 1, cannot be consumed directly by downstream applications as they need to be digitized first. Converting the scanned raster logs (PDFs or images) into CSV or LAS files is typically performed by a raster digitization system. Downstream applications of digitized raster data include well log correlation, prediction of missing log values, and extracting domain-specific inference from log data. Recently, Artificial Intelligence (AI)-based raster digitization systems have evolved, utilizing ML/ DL models to digitize the raster logs (Gune et al. (2023)). While there are numerous methods present in the literature on digitizing raster data, all of them involve multiple DL/ML models due to the complex nature of the task.

In this work, we build upon our previous work on a raster digitization system (Gune et al. (2023)), which includes three DL models: a raster segmentation model, a log header segmentation model, and a curve segmentation model, and one ML model for depth track processing. The goal of the raster segmentation model is to segment three important raster regions—the log track area, depth track area, and log header area —from the input raster image. The raster segmentation model is a DL model trained on labelled data using a supervised learning framework. Different segments of the raster, as extracted by the raster segmentation model, are further sent to different DL models for processing. The log header segmentation model takes the segmented log header as input and outputs the metadata of each recorded log, such as line style, unit, scale, and property name. The log header segmentation model is a DL model trained on the labelled data of log headers using a supervised learning framework. Moreover, it also uses OCR to extract the text corresponding to log properties. The curve segmentation model is a DL model trained on labelled data using a supervised learning framework. The goal of the curve segmentation model is to extract the curve pixels from the log track area and assign them values based on log header metadata. The curve segmentation model takes the

extracted log track area image as input (based on the output of raster segmentation model) and outputs a binary mask image corresponding to the curve. This binary mask is then later digitized by assigning values to each curve pixel using scale information from the log header metadata. The depth track processing model is an unsupervised model which extracts the depth numbers from the depth track and assigns depth values to every curve pixel.

Every model of the above raster digitization system is trained independently on the corresponding training data. Overall, the raster digitization system delivers robust performance when test data for each model is similar in distribution to the training data of the corresponding model. Note that the DL models are in cascade, resulting in the dependency of one model on the output of the previous DL model. If one of the DL models performs poorly, it results in the overall degradation of the raster digitization output. Alternatively, if the raster digitization system deployed in production performs poorly, the end-user or the developers may not be able to identify the real cause behind the poor performance. Identifying distribution shifts in data from training to testing stages is a complex task, especially for end-users who may lack technical expertise. Additionally, workflow developers may not have access to customer data due to privacy and data residency constraints, making it challenging to assess model performance on new data. It is well known that a DL model performs poorly on test data if the test data distribution differs significantly from the training data distribution. To improve the performance of a DL model, a universal solution of retraining or fine-tuning on (part of) test data is recommended. However, in the case of complex systems such as raster digitization, identifying the model(s) which performed poorly on the test data is a non-trivial task due to the above-mentioned challenges. In this work, we propose a novel inference-time approach to identify the DL model(s) whose performance is degraded due to data distribution drift. The proposed approach is fully automated and does not require any manual intervention in deciding which models need retraining. We highlight the important novel contributions of the work as below:

1. To the best of our knowledge, this is the first methodology designed to address the critical question of when to retrain model(s) in a complex raster digitization workflow containing multiple DL models.
2. The proposed approach does not introduce additional DL models or added complexity within the existing raster digitization workflow. Instead, it leverages simple, threshold-based classifiers that rely on intermediate outputs from the raster digitization workflow and require no additional training.
3. The method utilizes a common ensemble theme across three key models within the raster digitization workflow to construct task-specific retraining decision classifiers.
4. The proposed framework automates the decision-making process entirely, eliminating the need for manual intervention in determining retraining requirements.
5. The automation process respects data residency constraints, ensuring that retraining decisions can be made without compromising user data sovereignty or requiring external data transfer.

Once the DL model(s) which need retraining are identified, they can be retrained using separate workflows. How to retrain the DL model(s) is out of the scope of this work.

Related Work

Raster digitization workflow is predominantly inspired from object detection, instance segmentation, and semantic segmentation tasks on natural images. Numerous studies have focused on improving semantic and instance segmentation for typical computer vision datasets and natural images. We'll highlight a few key examples here. Long et al. (2015) pioneered fully convolutional networks (FCNs), a powerful DL method for end-to-end pixel-wise classification in semantic segmentation. Building on this, Chen et al. (2018) introduced DeepLab, which used dilated convolutions and atrous spatial pyramid pooling to capture multi-scale context, achieving high accuracy and efficiency. More recently, Generative Adversarial Networks (GANs) (Goodfellow et al., 2014) and conditional GANs (c-GANs) have been leveraged for semantic and

instance segmentation. Goodfellow et al. (2014) established GANs as a framework for training generative models. Expanding on this, Isola et al. (2017) developed pix2pix, a c-GAN-based architecture for image-to-image translation that showed impressive results, including in semantic segmentation. Zhu et al. (2017) further extended pix2pix with cycle-consistency loss for unpaired image-to-image translation. For instance segmentation, He et al. (2017) presented Mask R-CNN, which combined region-based convolutional neural networks (R-CNN) with an instance-level segmentation branch, achieving state-of-the-art performance in object detection and segmentation.

Convolutional neural networks (CNNs) have been remarkably successful in various image segmentation tasks, including those involving raster data. Existing raster digitization techniques often require manual intervention (Naseem et al., 2022; Neuralog, 2022). Witte et al. (2018) proposed a hybrid approach combining manual and automated processes. Our work on retraining decision system is based on our previous work on raster digitization workflow (Katole et al. (2025), Gune et al. (2024), M.S. Shakeel (2024), Katole et al. (2022)). While there are few works on retraining framework for raster digitization (Gune et al. (2024)), this is the first work to the best of our knowledge which provides an automated framework for retraining decision system for complex raster digitization workflow.

Methodology

In this section, we describe in detail the methodology to identify the DL model(s) of the raster digitization system which require retraining due to data distribution shift. We start with a detailed explanation of the workflow for raster digitization, which includes multiple DL/ML models. The proposed solution is built upon our previous work on raster digitization (Gune et al. (2023)).

Raster Digitization Workflow

A raster consists mainly of three sections as shown in Fig.1: 1) Log track area containing the logs (curves for property such as porosity, resistivity, gamma ray), 2) Log header containing metadata such as line style, unit, scale, name of the log property recorded in log track area, 3) Depth track containing the depth values measured against well logs recorded. A typical raster digitization workflow consists of 4 stages as shown in Fig 2.

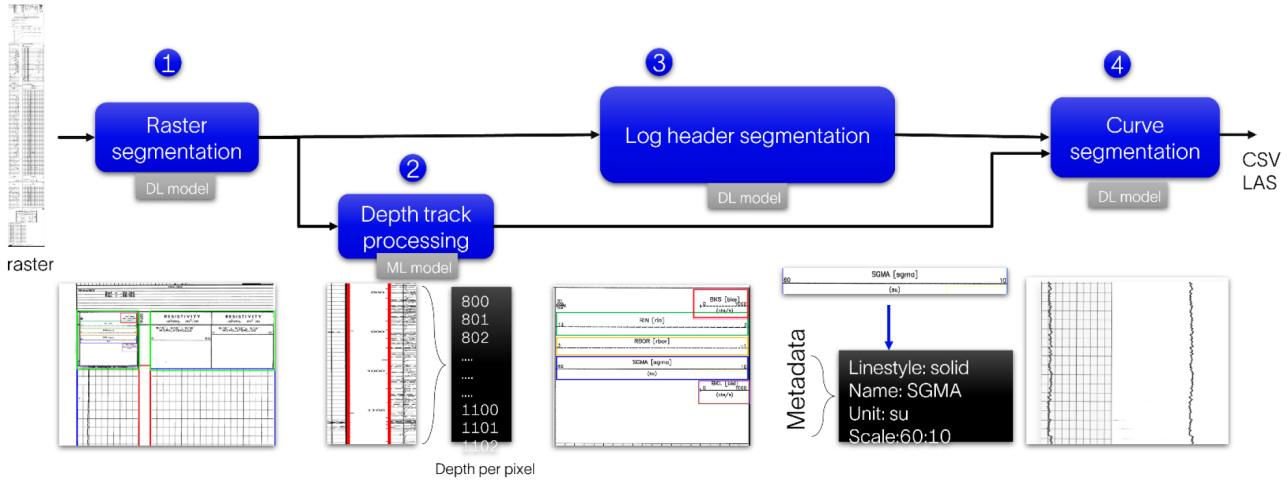

Figure 2—Raster digitization workflow from (Gune et al. (2023)).

Raster segmentation

The raster segmentation task aims at identifying three important raster regions: the log track area, log header area, and depth track area. The DL model used to perform this task is trained in a supervised learning manner. The dataset required for training the model consists of raster image tiles with ground truth mask

images representing different raster regions present in the raster image tile. Deeplabv3 (Chen et al. (2017)) is used to train the raster segmentation model, which outputs the segmentation masks for the three important raster regions.

The raster segmentation task is performed using a two-step approach to overcome the challenges of processing very long raster images, limited compute capabilities, and imbalanced data issues (Gune et al. (2023)). In the first step, coarse four-class segmentation is performed to identify regions corresponding to the log track area, log header area, depth track area, and background area. In the second step, which is a finer segmentation step, two separate models are trained to perform binary segmentation tasks, one for each log header region and log track region. At the end, outputs from the coarse and fine segmentation models are consolidated to provide the refined final segmentation mask.

Log header segmentation

The goal of the log header segmentation task is to identify and extract important metadata from the log header region. The log header region image, identified by the previous raster segmentation model, is processed to identify different instances of logs along with their properties such as name, line style format, unit, and scale. A DETR (Carion et al. (2020)) based DL model is used to identify the log instances and log properties. This task is formulated as instance segmentation and object segmentation, and the model is trained on the labeled dataset using a supervised learning framework to output the bounding boxes for each log instance and log property.

Curve segmentation

The core of the raster digitization workflow is to extract the log curves from the raster image. This task is achieved by training a Vision Transformer model (M. S. Shakeel (2024)) on a large-scale dataset. The dataset consists of log track region images with a binary image mask as the label. The binary image mask, which is the same size as the log track region image, consists of curve pixels as the foreground region pixels and non-curve pixels as the background region pixels. The DL model for curve segmentation is trained to output a binary mask where curve pixels corresponding to log data are present.

Depth track processing

This module in the raster digitization workflow aims at identifying the depth values in the depth track region, as provided by the raster segmentation task. The goal is to associate every log pixel with its corresponding depth value. As the number of depth values is not explicitly mentioned for each log curve point, this module interpolates the depth data based on the available depth values. This task is carried out using an unsupervised learning framework and does not require any training.

When to retrain the models of raster digitization workflow?

As discussed above, training-testing data distribution shifts degrade the overall raster digitization performance. If the raster digitization system is deployed in production, developers typically have limited or no access to the test data, considering security and data residency aspects. Hence, in such scenarios, it becomes challenging to understand which one or more models are underperforming. Note that each DL model for raster segmentation, log header segmentation, and curve segmentation tasks processes different data. It may be possible that overall raster data in the test set differs from the training data. However, one or a few of the models may still perform well on their respective data, considering the different nature and complexity of the data. In such cases, it is imperative to identify which models need to be retrained. The proposed solution, as shown in Figure 3, provides an automated framework to identify the DL models of the raster digitization workflow that are underperforming in their respective tasks. The novel solution does not require any additional models but is based on tapping the intermediate signals of the existing raster digitization workflow. For each of the three DL models mentioned above, we build a retraining decision

classifier (RDC) (training-free, rule-based) which signals if retraining is needed for the respective DL model. The overall theme for the retraining decision classifier is based on ensembling, which is described in detail in subsequent sections. The retraining framework shown in Figure 2 is out of scope of this work.

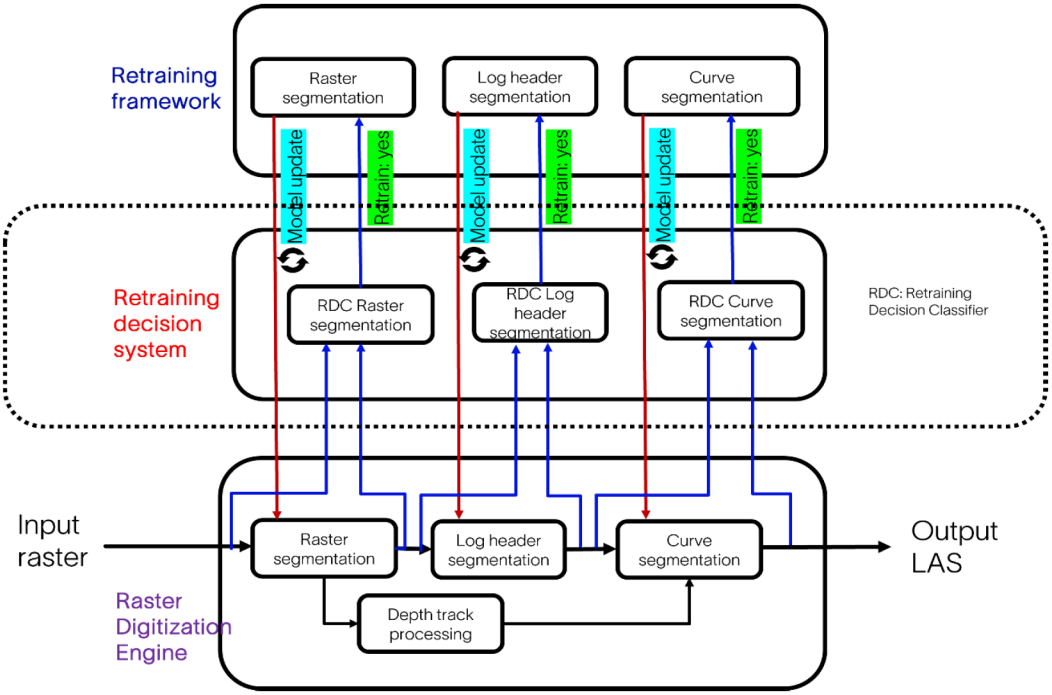

Figure 3—A workflow shows an automated framework for raster digitization retraining decision system.

Raster segmentation RDC

We revisit the challenges in raster segmentation and a two-step approach to address these. First, the highly skewed aspect ratio of raster images makes them unsuitable for direct processing using DL models due to constraints on computing power, as well as loss of resolution when resizing highly skewed raster images during processing. We proposed tile-based raster processing in our previous work (Gune et al. (2023)) to address these challenges. Second, raster image tiles contain imbalanced proportions of log track regions, depth track regions, and log header regions. While log track and depth track regions represent the majority of the spatial area in a raster image, log header regions are minority regions which occupy a relatively smaller area in the raster. To improve the segmentation of the minority class log header region, we proposed an ensemble approach for log header region segmentation and a simpler binary segmentation task.

Fig. 4 shows the two-step approach to the raster segmentation task, where three different models are used for coarse (a single multi-class segmentation model) and finer segmentation (two binary segmentation models). The raster is split into smaller tiles with a pre-defined height-to-width aspect ratio. Every tile is processed using two steps: in the first step, coarse segmentation for three regions—viz., log track, log header, and depth track—is performed. In the second step, two separate models are used for binary segmentation tasks, which segment the log track and log header regions separately. Based on the coarse segmentation for the log header region in the first step, raster image tiles are created from the original raster image such that (approximately) the entire log header region appears in one tile. For every tile in which a coarse log header is detected, three tiles are created by adjusting the tile boundaries of the full raster image such that: 1) the coarse log header region appears at the centre of the tile, 2) the coarse log header region appears towards the top of the tile, and 3) the coarse log header region appears towards the bottom of the tile. Each tile is processed using a binary segmentation model to output a binary mask for the log header region. Outputs of the three tiles are combined to create the final mask for the log header region.

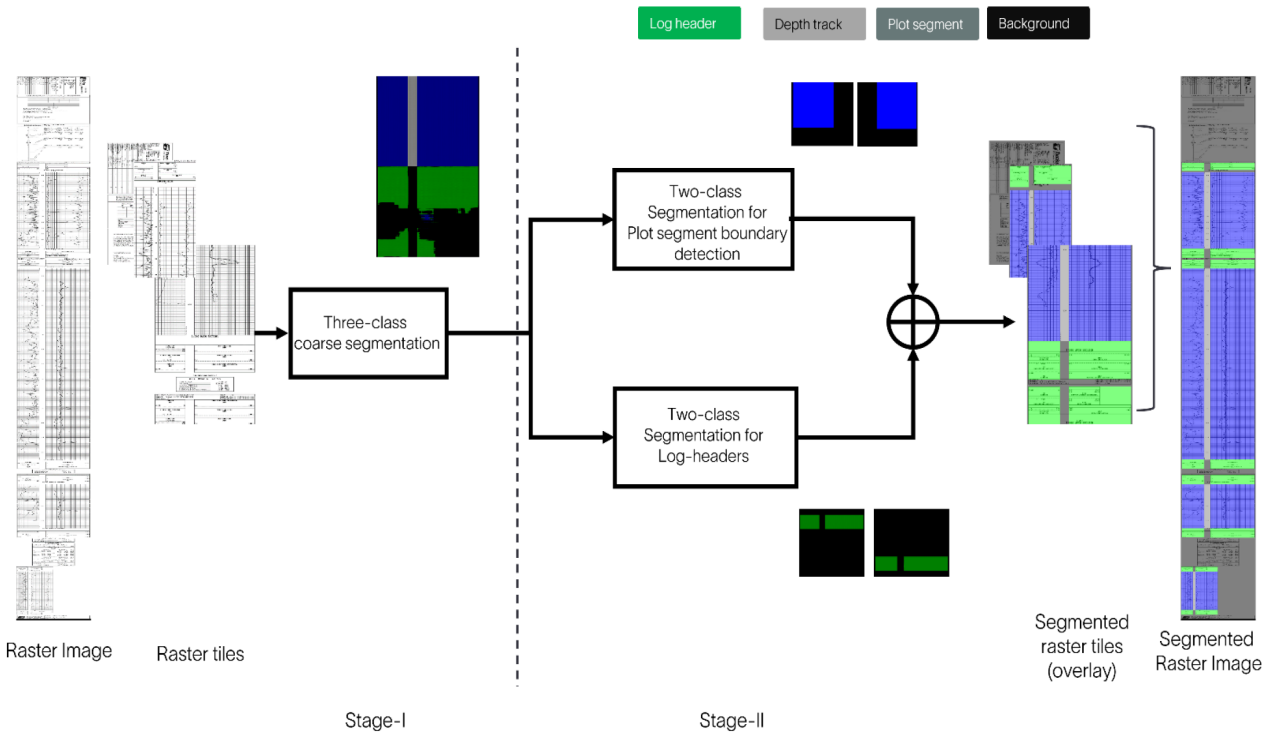

Figure 4—A workflow for two stage raster segmentation method from (Gune et al. 2023). Best viewed in colors.

Identifying the accuracy of the raster segmentation task on test data is a non-trivial task because: 1) it depends on how each of the two stages performs its corresponding segmentation tasks, and 2) as the learning task at hand needs to identify the class of every pixel in the image, it could be challenging to assess and aggregate the confidence at the pixel level. We propose a novel solution which uses the above challenges as a useful tool in building a retraining decision classifier for the raster segmentation task. Specifically, we utilize the outputs from both stages and use the pixel-level classification data from the output masks.

Experimentally, it is observed that the performance of raster segmentation tasks is correlated with the performance of the log header binary segmentation task. In turn, this depends on how accurately the minority class log header region is detected in the raster tile. This is because the majority class regions, such as log track and depth track regions, are typically segmented out correctly. This observation is consistent with the fact that higher uncertainty is associated with smaller data volumes in the case of log header regions. We use this fact and design our raster segmentation RDC based on the uncertainty of the log header region segmentation.

For uncertainty estimation, we compare the results from the first stage and second stage of raster segmentation (Fig. 5). Specifically, we analyze the segmentation masks for only the log header region in the first stage (coarse) and in the second stage (finer). It is experimentally observed that when the testing data is different from the training data, the model exhibits high uncertainty in the segmentation task. Large variations are observed in the output masks of the log header region generated by the first and second stage segmentation models. We use the variations in first and second stage mask output as a proxy for uncertainty estimation of the raster segmentation model.

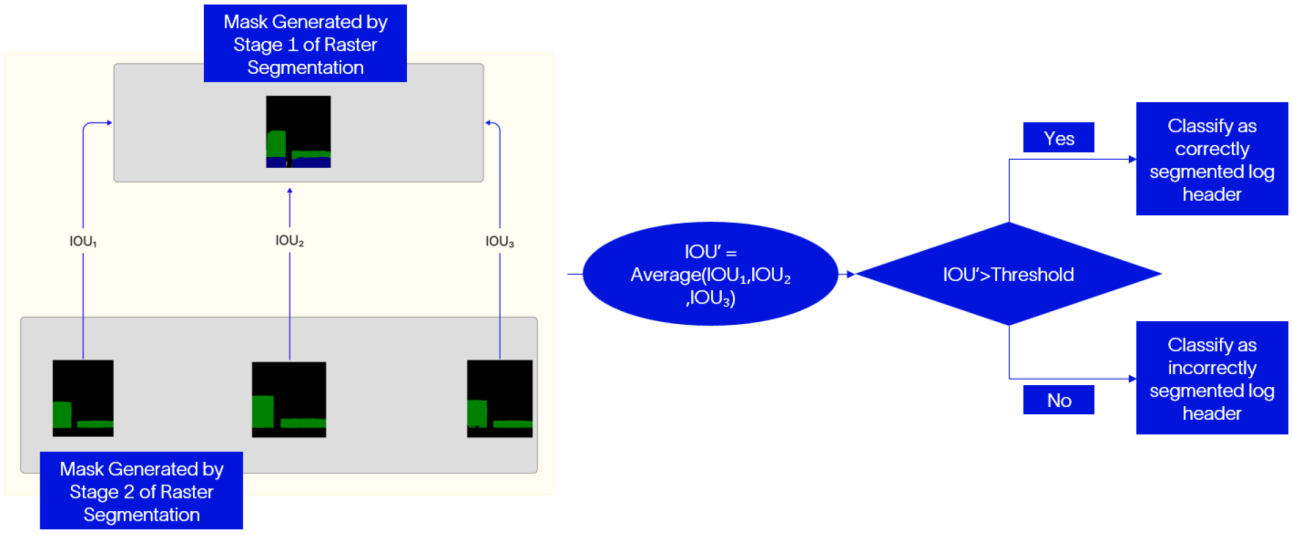

Figure 5—Figure shows the workflow for raster segmentation retraining decision classifier.

Fig. 5 shows the workflow for raster segmentation RDC. Note that the first stage provides the mask for coarse segmentation of the log track, log header, and depth track regions. The mask is processed to create a binary mask where the log header region is considered the only foreground region, while other regions, including the depth track and log track, are considered background. As mentioned above, in the second stage, for every tile, three tiles are obtained by varying the position of the (approximate) log header region within a tile. Therefore, the second stage processes the same log header region three times with variations in its position. We calculate the Intersection over Union (IoU) of the masks obtained in the first and second stages. Specifically, for every one of the three tiles in the second stage, we calculate the IoU of that tile's mask and the mask from the first stage. Note that the output masks from the second stage are appropriately shifted back to match the corresponding image locations in the mask from the first stage. Our final IoU is an average of the three IoU metrics.

When the IoU is large, it indicates that different models (coarse and finer) have similar responses regarding log header region detection. This also highlights that the spatial shift in the log header regions during second-stage processing has a minimal impact on log header region segmentation. For training data, it is expected that models from the first and second stages will respond in a similar way to detect the log header region. On the other hand, when the testing data distribution differs from the training data distribution, the first and second models are expected to behave differently for the same log header region. Moreover, as the model is not exposed to test data (which is different in distribution from the training data), the second-stage model may output different masks for three tiles due to variations in position. Such a scenario results in different log header masks generated by the first and second stages, leading to a lower IoU value. We propose the raster segmentation RDC based on the IoU value. A higher IoU value corresponds to lesser uncertainty in segmenting the log header region using different models (coarse and finer segmentation models) and different data (change of tile position in the second stage). A lower IoU value corresponds to higher uncertainty in the segmentation of the log header region. The higher uncertainty scenario may be flagged as a need for retraining the raster segmentation model. The threshold of the RDC is set based on a validation dataset.

To summarize, the proposed novel approach uses the principle of ensembling over models (coarse and finer segmentation models) and ensembling over the data (shift of tile positions in the second stage) to provide a proxy for the uncertainty estimation in minority class log header segmentation. It is important to note that no new ML/DL models are used in identifying whether retraining is required for a raster

segmentation task. We use the existing signals and outputs in the raster segmentation framework to design the RDC.

Log header segmentation RDC

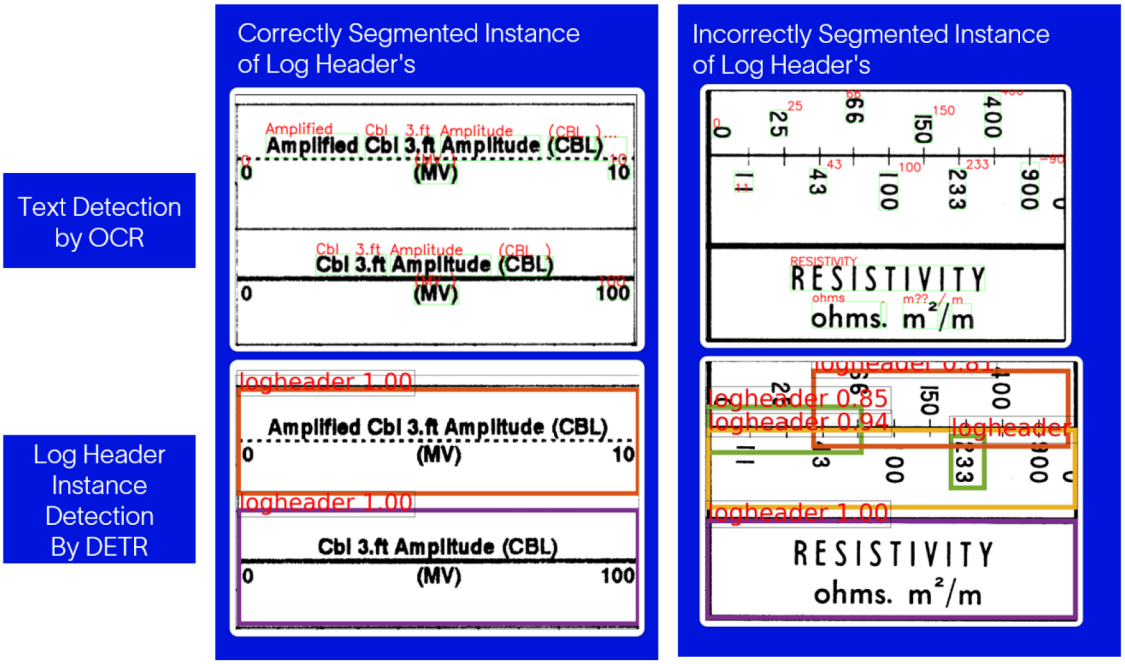

Figure 6—An example illustrating the correct and incorrect log header segmentation. Best viewed in color.

The goal of log header segmentation is to accurately segment different instances and properties of logs from the log header region. The input to log header segmentation is the log header image (as identified by the raster segmentation model), and the output contains the bounding boxes for each log header instance and its corresponding log properties, such as log name, scale, unit, and linestyle (Fig. 1).

The log header segmentation task is performed by training a DETR (Carion et al. (2020)) model which outputs the bounding boxes for the log instances and their corresponding log properties. Once the bounding boxes for the log properties are identified, an Optical Character Recognition (OCR) engine is used to extract the actual log properties like scale, unit, name, etc.

Although a DETR-based model can provide confidence on the detection of bounding boxes, this may not be sufficient for identifying model performance. This is because log headers often contain multiple objects overlapping with each other; for instance, log properties such as name, scale, and unit reside within the bounding box of a log instance. While the detection of log properties could have very high confidence, the corresponding log instance detection may have low confidence. In this complex setting, aggregating the confidence scores for different detections and associating them with the need for retraining the model can be challenging. In our proposed solution, we do not depend on the model confidence score for different detections but utilize signals from OCR data and the bounding boxes of detections.

We note that there are two important aspects of the log header segmentation task. The first is the accurate detection of bounding boxes for different log instances, and the second corresponds to the accurate detection of bounding boxes for the log properties. It is clear that if the bounding boxes for all log properties are correctly detected, they are expected to lie inside the bounding box of that log instance. We use this fact and propose a simple but effective solution to identify whether the log header segmentation model is performing accurately on test data.

Fig. 7 shows the workflow for log header segmentation RDC. We propose to perform OCR on the entire log header region instead of on the image regions identified by different bounding boxes obtained from the DETR model. Typically, off-the-shelf OCR engines are fairly accurate in terms of text and text position detection. For every detected text in the log header region, we associate it with one or more detected log instances (bounding boxes). If a text (log property) bounding box resides partly within more than one log instance bounding box, it corresponds to incorrect log instance detection (example on the right in Fig.6). This scenario represents a one-to-many mapping of text to log instance. Moreover, if a text bounding box does not reside inside any of the detected log instances, it corresponds to a missed log instance detection. This scenario represents a one-to-none mapping of text to log instance. If the text bounding box is completely residing inside one log instance, it corresponds to correct log instance detection. This scenario represents a one-to-one mapping of text to log instance (example on the left side in Fig. 6). We sum the one-to-many and one-to-none instances, which represent incorrect detections, and the one-to-one instances as correct detections. The proposed log header RDC is based on the ratio of correct to incorrect detections. A high ratio indicates that the log header segmentation model performs well on the test data, while a lower ratio indicates poor log header detection, necessitating the retraining of the DETR model.

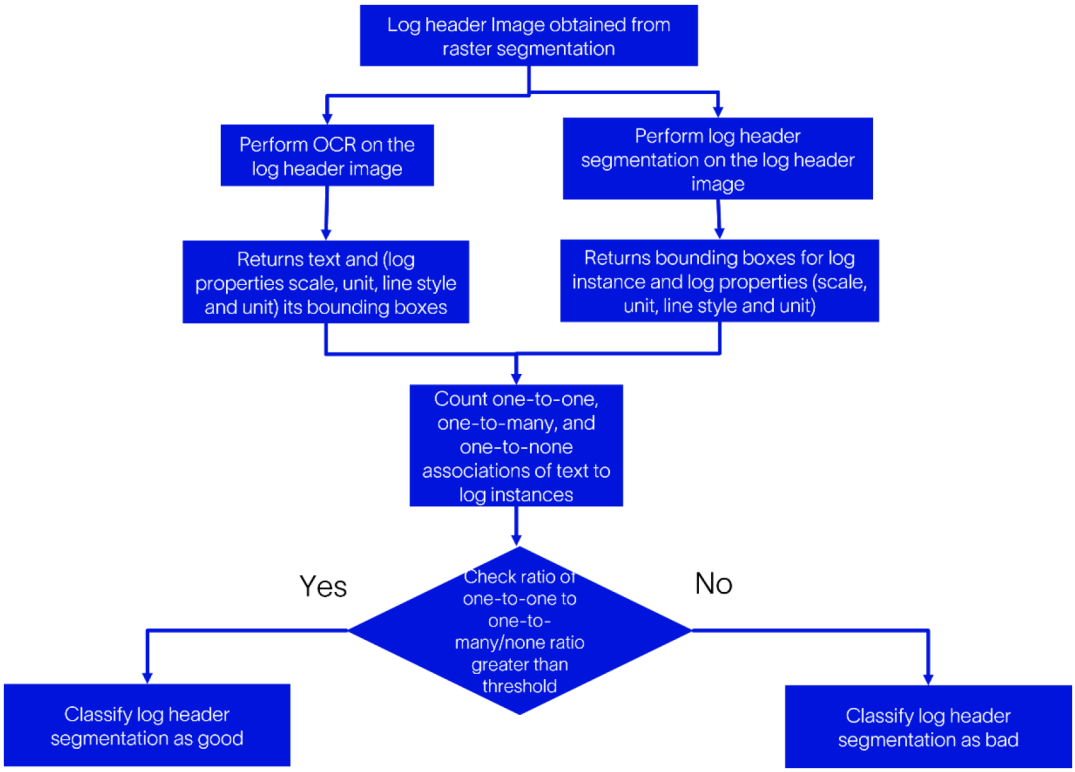

Figure 7—Workflow for log header segmentation RDC.

It is important to note that a simple flip in the sequence of operations can provide more insights into log header segmentation performance. In one sequence of operations, the log header segmentation model involves DETR-based detection of bounding boxes followed by performing OCR on the detected bounding boxes. In another sequence of operations, OCR is performed on the entire log header region image, and the detected text is associated with the bounding boxes detected by the DETR model. Here, no additional ML/DL model is used in identifying the need for retraining of the log header segmentation model. To summarize, the proposed log header segmentation RDC is based on the ensembling of sequences of operations within the log header segmentation task.

Curve segmentation RDC

The goal of the curve segmentation model is to extract curve pixels from the log track area. The input to this model is a log track region image (identified by the raster segmentation model) augmented with a line style image from the log header region. The line style image from the log header region is obtained using bounding box information from the log header segmentation model. The output of the curve segmentation model is a binary mask with foreground pixels corresponding to the log curve. The curve segmentation model is trained using ViT (Dosovitskiy et al. (2020)) on a labelled dataset (M.S. Shakeel (2024)).

While curve segmentation is at the core of the raster digitization workflow, it poses numerous challenges in identifying if the model performance on test data is within acceptable limits. There are multiple scenarios where curve segmentation performance may deteriorate, such as: 1) only one of multiple log curves is extracted, or 2) log curves are extracted partially. In the absence of ground truth, it's clear that these scenarios cannot be distinguished from instances where the curve segmentation model accurately extracts the curve pixels. To overcome this, we propose a novel and effective solution that uses information from the output masks and input log track region image to build a curve segmentation RDC.

Fig. 8 shows the workflow of curve segmentation RDC. We use output masks from the curve segmentation model and log track region images to build the decision classifier. Note that the curve segmentation model outputs a separate binary mask for each log curve (M.S. Shakeel (2024)). We combine all binary masks such that the final mask contains foreground pixels belonging to all log curves. We call this mask M1.

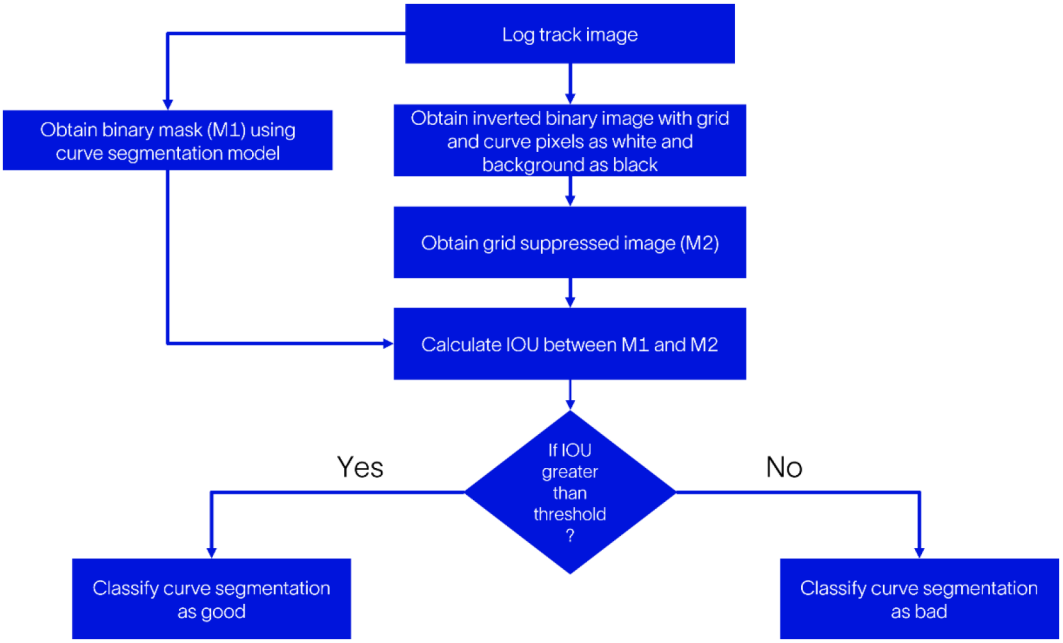

Figure 8—Workflow for the curve segmentation RDC.

We use the input log track region image and binarize it. This binarization results in an image containing grid and curve pixels of the log data while removing noise. We then invert the image, turning black pixels white and white pixels black. In the inverted image, both grid and curve pixels are white. We then calculate the sum of white pixels in the image along the length axis and plot it as a graph of pixel location on the length (X) axis versus the sum of white pixel count on the Y axis (Fig. 9). As can be observed in the image, the plot contains periodic spikes corresponding to grid pixels, with lower and irregular spikes corresponding to curve pixels.

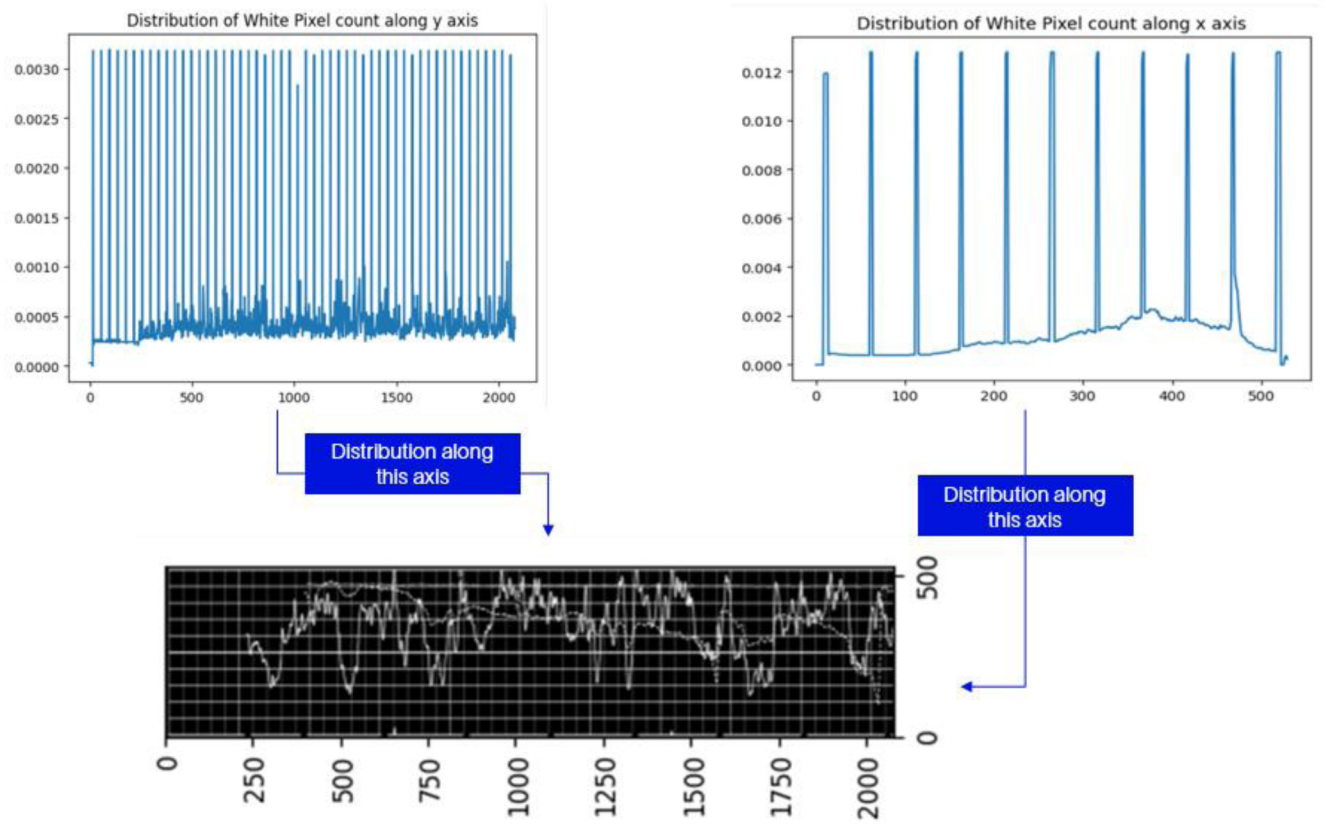

Figure 9—Illustration showing the distribution of white pixels for a plot segment along x and y axes.

We use Otsu's threshold method to filter the spikes corresponding to grid lines while retaining the spikes corresponding to curve pixels. We identify the positions of the pixels retained after Otsu's thresholding and retain only those pixels in the inverted binary mask. We repeat a similar process for the other axis of the inverted image. This processing results in a mask that predominantly contains white pixels corresponding to log curve data. We call this mask M2 which is grid suppressed log track region image.

Note that obtaining mask M2 is a different approach to performing the curve segmentation task. While this is a non-learning-based method, it can collectively extract pixels from all log curves, though it cannot label curve pixels for different log curves.

Returning to Fig. 8, we calculate the IoU for masks M1 and M2. A large IoU value indicates that M1 and M2 contain a significant number of overlapping pixels for curve data. This means the curve segmentation model is essentially able to accurately extract pixels from all log curves. On the other hand, a smaller IoU value reflects that a large number of pixels from the log curves are missing in the output mask given by the curve segmentation DL model.

We propose the curve segmentation RDC based on the IoU value. If the IoU value is below a threshold (obtained from the validation dataset), retraining of the curve segmentation model is required. It's worthwhile to note that the above decision classifier is also based on an ensemble of different curve segmentation methods.

Experiments

Our training dataset for the raster digitization workflow consists of three separate labelled datasets for raster segmentation, log header segmentation, and curve segmentation tasks. The three RDCs for the raster segmentation, log header segmentation, and curve segmentation models are separately evaluated on their respective test datasets. We analyse the performance of the proposed RDCs for each segmentation task across three different datasets, which are distinct from the training dataset used to train the DL model.

Fig. 10 shows the results for the raster segmentation RDC in the form of a confusion matrix. As discussed earlier, we analyse the raster segmentation task's performance based on how well the log header region segmentation performs. Each test sample is manually evaluated based on the output log header region mask provided by the DL model and assigned a "good" or "bad" segmentation label. This essentially pertains to creating the labelled dataset for the task of knowing when to retrain the raster segmentation model. During inference, the test sample is passed through the RDC, which yields the IoU value for output masks of log header regions from the first and second stages of the raster segmentation framework. If the IoU is above the threshold (set at 80% based on the validation dataset), the raster segmentation results for the test sample are considered "good"; otherwise, they are considered as "bad." Based on the results from all samples, we note that the RDC accuracy is 66.0%, 76.1%, and 81.2% for Dataset 1, Dataset 2, and Dataset 3, respectively. This demonstrates that the RDC is accurate in identifying the need to retrain the raster segmentation model.

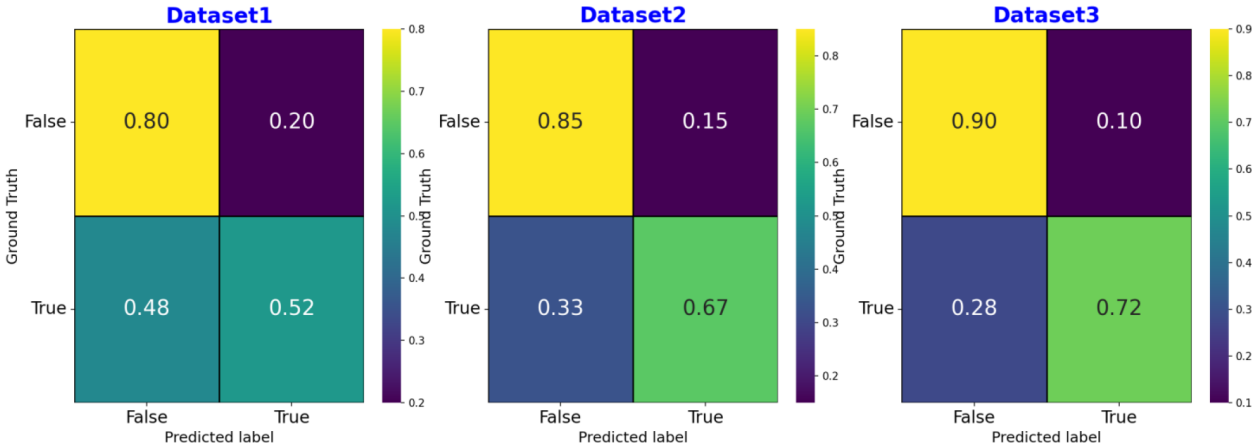

Figure 10—Figure shows the performance of raster segmentation RDC on three different datasets. True labels correspond to correctly segmented log header region while false labels correspond to incorrectly segmented log header region. The balanced class accuracy over the three datasets is 66%, 76.1%, and 81.2%. Best viewed in color.

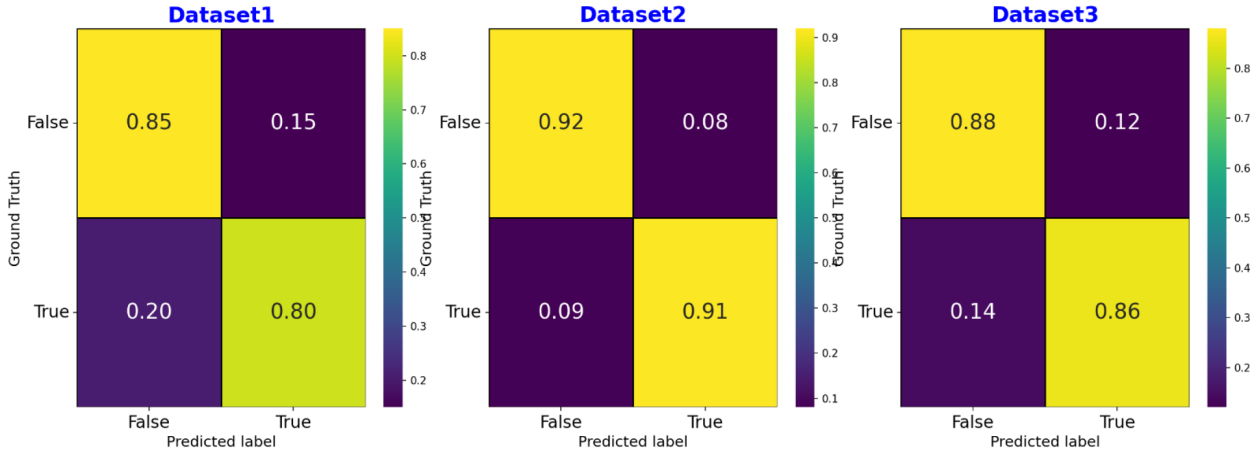

Figure 11—Figure shows the performance of log header segmentation RDC on three different datasets. True labels correspond to correct bounding boxes for log instances and their properties while false labels correspond to incorrectly detected bounding boxes for log instances and their properties. The balanced class accuracy over the three datasets is 82.3%, 91.7%, and 86.9%. Best viewed in color.

Fig.11 shows the results for the log header segmentation RDC in the form of a confusion matrix. This RDC essentially identifies whether the bounding boxes of log property text regions are associated with their corresponding log instance bounding boxes based on spatial locations. Each test sample is manually evaluated based on the results of the log header segmentation model to create the labelled dataset for the

task of knowing when to retrain the log header segmentation model. If the bounding boxes for all log header instances, along with their properties such as name, scale, unit, and line styles, are identified and accurately associated with each other, such an output is marked as "good segmentation." If there is one or more incorrect associations of the bounding boxes of log properties with their instances, the results for such samples are marked as "bad segmentation." Each test sample is evaluated using the RDC. As described in the previous section, the RDC marks the output as "good" or "bad" depending on the association of bounding boxes of log instances and their properties. It can be observed that the RDC exhibits excellent performance with an accuracy of 82.3%, 91.7%, and 86.9% on Dataset 1, Dataset 2, and Dataset 3, respectively.

Fig.12 shows the results for the curve segmentation RDC. This RDC identifies the overlap of curve pixels between masks M1 and M2, which are extracted from the log track region using two different approaches. We manually evaluate the curve segmentation of each test sample and mark it as "good" or "bad" segmentation depending on the quality of curve pixel extraction. Subsequently, each test sample is then passed through the decision classifier, which marks the curve segmentation output as "good" or "bad" depending on whether the IoU score is above or below the threshold (a value set to 70% based on the validation dataset). We note that for the challenging curve segmentation task, the curve segmentation RDC demonstrates excellent accuracies of 75.6%, 75.1%, and 77.0% on the three datasets.

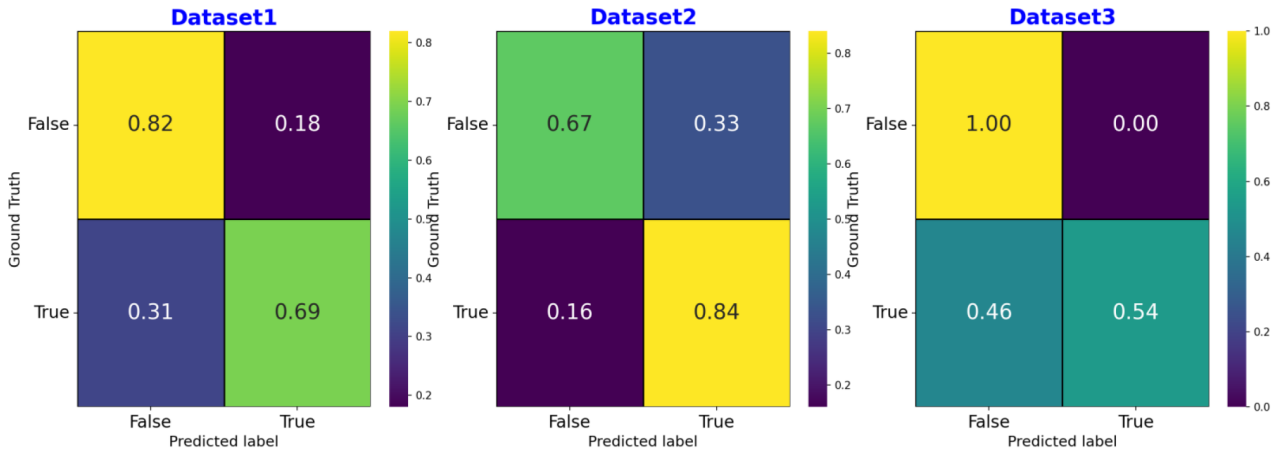

Figure 12—Figure shows the performance of curve segmentation RDC on three different datasets. True labels correspond to correctly segmented log curve while false labels correspond to incorrectly segmented log curve. The balanced class accuracy over the three datasets is 75.6%, 75.1%, and 77%. Best viewed in color.

Conclusions

In this work, we proposed a solution to address the non-trivial question of when to retrain one or more DL models within a complex raster digitization workflow. The raster digitization workflow is a complex task because it contains multiple ML/DL models in cascade or in parallel. Moreover, each task has different complexities in terms of data and the model itself. When such a raster digitization workflow deployed in production exhibits poor performance, it becomes challenging to understand which DL models, or combination of models, are underperforming. Due to data access restrictions, it can further become a tedious task for developers to understand the root cause behind poor performance. In such situations, an automated decision system is essential to detect poorly performing DL models. Furthermore, it is advisable not to increase the ML/DL model complexity or add extra models to achieve this automated decision system. The proposed solution addresses the above challenges and exhibits robust performance on diverse datasets. As a future direction, we would like to test the RDCs on additional diverse datasets and further improve their accuracy.

References

Chen, Liang-Chieh, George Papandreou, Florian Schroff, and Hartwig Adam. "Rethinking atrous convolution for semantic image segmentation." arXiv preprint arXiv:1706.05587 (2017).

Carion, Nicolas, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. "End-to-end object detection with transformers." In European conference on computer vision, pp. 213–229. Cham: Springer International Publishing, 2020.

M. S. Shakeel. "Conditional Curve Digitization for Raster Segmentation" In *fourth EAGE Digital*, pp.1–5. 2024.

Katole A.,L. Mangsuli, P., Gune, O., A., M.,S., Shakeel, Abhiman, N., Aria, Abubakar, "Deep learning based raster digitization system", In *second IMAGE*, 2022.

Gune, O., A., Lokhande, A, Mangsuli, P., "On raster image segmentation, well log instance detection, and depth information extraction from rasters using deep learning models", In Abu Dhabi International Petroleum Exhibition and Conference 2023.

Katole, A., L., Gune, O., A., Mangsuli, P., "Rejuvenating legacy data by digitizing raster logs", *Artificial Intelligence for Subsurface Characterization and Monitoring*, Elsevier, 2025.

Gune, O. A, Mangsuli, P., "An efficient retraining framework for raster digitization", In *fourth EAGE Digital* 2024.

Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani et al "An image is worth 16×16 words: Transformers for image recognition at scale." arXiv preprint arXiv:2010.11929 (2020).

Girshick, R. (2015). Fast r-cnn. In Proceedings of the IEEE international conference on computer vision (pp. 1440–1448).

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., … Bengio, Y. (2014). Generative Adversarial Networks. In *Advances in Neural Information Processing Systems (NIPS)*.

He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask r-cnn. In Proceedings of the IEEE international conference on computer vision (pp. 2961–2969).

Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. (2017). Image-to-Image Translation with Conditional Adversarial Networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Long, J., Shelhamer, E., & Darrell, T. (2015). Fully convolutional networks for semantic segmentation. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 3431–3440).

Nasim, M.Q., Patwardhan, N., Maiti, T., & Singh, T. (2022). Digitization of Raster Logs: A Deep Learning Approach. *arXiv preprint* arXiv:2210.05597.

Neuralog (2022). Retrieved from https://www.neuralog.com/

Wang, T.-C., Liu, M.-Y., Zhu, J.-Y., Tao, A., Kautz, J., & Catanzaro, B. (2018). High-Resolution Image Synthesis and Semantic Manipulation with Conditional GANs. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Zhu, J. Y., Park, T., Isola, P., & Efros, A. A. (2017). Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. In Proceedings of the IEEE International Conference on Computer Vision (ICCV).